Werkzeugverwaltungstool

Testprotokoll

Testdatum: 18.03.2026

Testumgebung Backend: pytest 8.4.2 · Python 3.9.6 · SQLite In-Memory

Testumgebung Frontend: Playwright · Chromium · React/Vite · API-Mocks

Gesamtergebnis: 226 bestanden · 0 fehlgeschlagen (127 Backend + 99 Frontend)

Laufzeit Backend: 17.87 s

Laufzeit Frontend: 1 min 30 s

Autorin: Maila Anna Pignari

Ausbildungsberuf: Fachinformatikerin für Anwendungsentwicklung

1 Überblick

Dieses Testprotokoll dokumentiert alle automatisierten Tests des Werkzeugverwaltungstools – 127 Backend-Tests (pytest/FastAPI) und 99 Frontend-End-to-End-Tests (Playwright/React). Alle 226 Tests wurden am 18.03.2026 ausgeführt und erfolgreich bestanden.

1.1 Backend-Tests (pytest)

ID	Testbereich	Gesamt	Bestanden	Fehlgeschlagen
TP-01	Authentifizierung	10	10	0
TP-02	Nutzer	16	16	0
TP-03	Abteilungen	14	14	0
TP-04	Werkzeuge & Kategorien	16	16	0
TP-05	Inventar / Exemplare	7	7	0
TP-06	Ausleihen	8	8	0
TP-07	Ausleiheanfragen	6	6	0
TP-08	Meldungen (Issues)	6	6	0
TP-09	Rollen	8	8	0
TP-10	Ausleihepositionen	6	6	0
TP-11	Anfragepositionen	6	6	0
TP-12	Anfrage-Status	6	6	0
TP-13	Werkzeugstatus	6	6	0
TP-14	Werkzeugzustände	6	6	0
TP-15	Meldungs-Status	6	6	0

1.2 Frontend-Tests (Playwright)

ID	Testbereich	Gesamt	Bestanden	Fehlgeschlagen
FE-01	Authentifizierung	8	8	0
FE-02	Dashboard & Navigation	11	11	0
FE-03	Werkzeuge	11	11	0
FE-04	Inventar / Exemplare	11	11	0
FE-05	Ausleiheanfragen	10	10	0
FE-06	Ausleihen	10	10	0
FE-07	Meldungen (Issues)	14	14	0
FE-08	Nutzer	12	12	0
FE-09	Abteilungen	12	12	0

2 TP-01: Authentifizierung

Quelldatei: src/test/test_auth.py | Anzahl Tests: 10

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_login_success	POST /auth/login email: admin@example.com password: admin123	HTTP 200 access_token im Body	HTTP 200 access_token erhalten	**PASSED**	JWT-Secret prüfen; Passwort-Hash in DB validieren; passlib-Konfiguration kontrollieren
2	test_login_wrong_password	POST /auth/login email: admin@example.com password: falsch	HTTP 401 Detail: Ungültige Anmeldedaten	HTTP 401 erhalten	**PASSED**	Passwort-Vergleichslogik in security.py prüfen; bcrypt-Version überprüfen
3	test_login_unknown_email	POST /auth/login email: unbekannt@x.de password: irgendwas	HTTP 401 Detail: Ungültige Anmeldedaten	HTTP 401 erhalten	**PASSED**	Nutzer-Lookup in DB prüfen; Email-Vergleich case-sensitiv?
4	test_protected_route_without_token	GET /users/ Kein Authorization-Header	HTTP 401 Detail: Not authenticated	HTTP 401 erhalten	**PASSED**	JWT-Middleware-Dependency in Route prüfen; Depends(get_current_user) vorhanden?
5	test_protected_route_with_token	GET /users/ Authorization: Bearer <gültiger Token>	HTTP 200 Nutzerliste zurückgegeben	HTTP 200 erhalten	**PASSED**	Token-Dekodierung in security.py debuggen; SECRET_KEY in .env prüfen
6	test_create_user_requires_admin	POST /users/ Bearer Token (Rolle: Mitarbeiter)	HTTP 403 Nicht autorisiert	HTTP 403 erhalten	**PASSED**	require_role(ADMIN_ID) in Route-Dependency prüfen; role_id im JWT-Token korrekt?
7	test_admin_can_create_user	POST /users/ Bearer Token (Rolle: Admin) Name, Email, Passwort, role_id	HTTP 201 Neuer Nutzer im Body	HTTP 201 erhalten	**PASSED**	Admin-Rolle im Token prüfen; Datenbankverbindung testen
8	test_logout_success	POST /auth/logout Authorization: Bearer <Token>	HTTP 200 Token in Blacklist	HTTP 200 erhalten	**PASSED**	Token-Blacklist-Mechanismus prüfen; In-Memory-Set korrekt initialisiert?
9	test_logout_token_rejected_afterwards	POST /auth/logout, dann erneut GET /users/ mit gleichem Token	HTTP 401 Token ungültig nach Logout	HTTP 401 erhalten	**PASSED**	Blacklist-Prüfung in get_current_user() sicherstellen
10	test_logout_without_token	POST /auth/logout Kein Authorization-Header	HTTP 401	HTTP 401 erhalten	**PASSED**	Dependency-Check in logout-Route überprüfen

3 TP-02: Nutzer

Quelldatei: src/test/test_users.py | Anzahl Tests: 16

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_user	POST /users/ first_name, last_name, email, password, role_id, department_id	HTTP 201 Nutzer ohne Passwort-Hash im Body	HTTP 201 erhalten	**PASSED**	FK auf role_id und department_id prüfen; Passwort-Hashing prüfen
2	test_get_users_list	GET /users/	HTTP 200 Nutzerliste	HTTP 200 erhalten	**PASSED**	DB-Query prüfen
3	test_get_user_by_id	GET /users/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
4	test_local_domain_email_serialized_correctly	POST /users/ email: user@intern (kein TLD)	HTTP 201 Email korrekt gespeichert	HTTP 201 erhalten	**PASSED**	Pydantic-Email-Validator: allow_reuse_validator oder EmailStr prüfen
5	test_invalid_email_rejected	POST /users/ email: kein-at-zeichen	HTTP 422 Validierungsfehler	HTTP 422 erhalten	**PASSED**	Pydantic EmailStr-Validierung in Schema prüfen
6	test_get_user_not_found	GET /users/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
7	test_duplicate_email_rejected	POST /users/ mit bereits vorhandener Email	HTTP 400 E-Mail bereits vergeben	HTTP 400 erhalten	**PASSED**	Unique-Constraint auf email prüfen; IntegrityError-Handling
8	test_update_user	PUT /users/1 first_name: Neuer Name	HTTP 200	HTTP 200 erhalten	**PASSED**	Update-CRUD prüfen
9	test_delete_user	DELETE /users/1	HTTP 204	HTTP 204 erhalten	**PASSED**	Kaskadierung prüfen; aktive Ausleihen vorhanden?
10	test_create_user_invalid_role_rejected	POST /users/ role_id: 9999	HTTP 400 oder 422 Ungültige Rolle	HTTP 400/422 erhalten	**PASSED**	FK-Constraint auf role_id prüfen
11	test_create_user_invalid_department_rejected	POST /users/ department_id: 9999	HTTP 400 oder 422 Ungültige Abteilung	HTTP 400/422 erhalten	**PASSED**	FK-Constraint auf department_id prüfen
12	test_update_role_from_manager_promotes_successor	PUT /users/<manager_id> role_id: EMPLOYEE (andere Mitarbeiter in Abt. vorhanden)	HTTP 200 Nachfolger automatisch Manager	HTTP 200, Nachfolger befördert	**PASSED**	Auto-Promote-Logik in update_user prüfen
13	test_update_role_to_manager_sets_department_lead	PUT /users/<mitarbeiter_id> role_id: MANAGER	HTTP 200 Abteilung bekommt neuen Leiter	HTTP 200, lead_user_id aktualisiert	**PASSED**	set_department_lead-Logik in update_user prüfen
14	test_update_manager_moves_to_new_dept	PUT /users/<manager_id> department_id: andere Abteilung	HTTP 200 Alte Abt. bekommt Nachfolger Neue Abt. hat neuen Leiter	HTTP 200 erhalten	**PASSED**	Cross-Dept-Move-Logik in update_user prüfen
15	test_create_department_manager_demotes_old_lead	POST /users/ role_id: MANAGER department_id: Abt. mit bestehendem Leiter	HTTP 201 Alter Leiter → Mitarbeiter	HTTP 201, alter Leiter demotiert	**PASSED**	Demote-Logik bei Manager-Erstellung prüfen
16	test_delete_lead_promotes_random_successor	DELETE /users/<lead_id> (andere Mitarbeiter in Abt.)	HTTP 204 Zufälliger Nachfolger wird Manager	HTTP 204, Nachfolger Manager	**PASSED**	Delete-User-Logik prüfen; random_successor korrekt?

4 TP-03: Abteilungen

Quelldatei: src/test/test_departments.py | Anzahl Tests: 14

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_department	POST /departments/ name: Testmontage	HTTP 201 Abteilung mit ID im Body	HTTP 201 erhalten	**PASSED**	Datenbankschema prüfen; unique constraint auf name?
2	test_get_departments_list	GET /departments/	HTTP 200 Liste aller Abteilungen	HTTP 200 erhalten	**PASSED**	DB-Session prüfen; Tabelle vorhanden?
3	test_get_department_by_id	GET /departments/1	HTTP 200 Abteilung mit id=1	HTTP 200 erhalten	**PASSED**	ID-Parameter-Parsing prüfen
4	test_get_department_not_found	GET /departments/9999	HTTP 404 Detail: Nicht gefunden	HTTP 404 erhalten	**PASSED**	404-Handler in Route prüfen; get_or_404-Logik testen
5	test_update_department	PUT /departments/1 name: NeuerName	HTTP 200 Aktualisierter Name im Body	HTTP 200 erhalten	**PASSED**	PATCH/PUT-Logik im CRUD prüfen; Commit fehlt?
6	test_delete_department	DELETE /departments/<id> (keine Mitglieder)	HTTP 204 Abteilung gelöscht	HTTP 204 erhalten	**PASSED**	Foreign-Key-Constraints prüfen; Kaskadierung konfiguriert?
7	test_delete_department_with_members_reassigns	DELETE /departments/<id> (Mitglieder vorhanden, andere Abteilung existiert)	HTTP 204 Mitglieder in andere Abt. verschoben	HTTP 204 erhalten	**PASSED**	Reassign-Logik im CRUD prüfen; Zielabteilung muss existieren
8	test_delete_department_demotes_lead	DELETE /departments/<id> (Abteilung hat Leiter)	HTTP 204 Leiter wird zum Mitarbeiter	HTTP 204 erhalten	**PASSED**	Demote-Logik prüfen; role_id korrekt gesetzt?
9	test_delete_last_department_with_members_rejected	DELETE /departments/<einzige_id> (Nutzer vorhanden)	HTTP 400 Letzte Abteilung kann nicht gelöscht werden	HTTP 400 erhalten	**PASSED**	Guard-Kondition in delete_department prüfen
10	test_duplicate_lead_user_rejected	PUT /departments/1 lead_user_id: bereits Leiter einer anderen Abt.	HTTP 400 Nutzer bereits Leiter	HTTP 400 erhalten	**PASSED**	Duplikat-Prüfung im CRUD validieren
11	test_set_lead_promotes_to_manager	PUT /departments/<id> lead_user_id: <mitarbeiter_id>	HTTP 200 Nutzer bekommt Rolle Manager	HTTP 200, Rolle aktualisiert	**PASSED**	Auto-Promote-Logik in update_department prüfen
12	test_replace_lead_demotes_old_manager	PUT /departments/<id> lead_user_id: <neuer_nutzer>	HTTP 200 Alter Leiter → Mitarbeiter	HTTP 200, alter Leiter demotiert	**PASSED**	Demote-Pfad in set_lead_user prüfen
13	test_remove_lead_demotes_to_employee	PUT /departments/<id> lead_user_id: null	HTTP 200 Leiter wird Mitarbeiter	HTTP 200 erhalten	**PASSED**	Null-Handling in update_department prüfen
14	test_remove_lead_auto_promotes_successor	PUT /departments/<id> lead_user_id: null (andere Mitarbeiter vorhanden)	HTTP 200 Nachfolger automatisch befördert	HTTP 200, Nachfolger Manager	**PASSED**	Random-Promote-Logik in CRUD prüfen; Mitarbeiterliste leer?

5 TP-04: Werkzeuge

Quelldatei: src/test/test_tools.py | Anzahl Tests: 16

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_tool_category	POST /tool-categories/ name: Elektro	HTTP 201	HTTP 201 erhalten	**PASSED**	Unique-Constraint prüfen
2	test_get_tool_categories	GET /tool-categories/	HTTP 200	HTTP 200 erhalten	**PASSED**	DB-Abfrage prüfen
3	test_get_tool_category_by_id	GET /tool-categories/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
4	test_get_tool_category_not_found	GET /tool-categories/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
5	test_update_tool_category	PUT /tool-categories/1 name: Handwerkzeug	HTTP 200	HTTP 200 erhalten	**PASSED**	Update-CRUD prüfen
6	test_delete_tool_category	DELETE /tool-categories/1	HTTP 204	HTTP 204 erhalten	**PASSED**	FK-Referenzen in Werkzeugen prüfen
7	test_create_tool	POST /tools/ name, category_id, description	HTTP 201 Werkzeug mit ID	HTTP 201 erhalten	**PASSED**	FK auf category_id prüfen
8	test_get_tools_list	GET /tools/	HTTP 200 Werkzeugliste	HTTP 200 erhalten	**PASSED**	DB-Query prüfen
9	test_get_tool_by_id	GET /tools/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
10	test_get_tool_not_found	GET /tools/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
11	test_update_tool	PUT /tools/1 name: GeänderterName	HTTP 200	HTTP 200 erhalten	**PASSED**	Update-Logik prüfen
12	test_delete_tool	DELETE /tools/1	HTTP 204	HTTP 204 erhalten	**PASSED**	Kaskadierung auf Exemplare prüfen
13	test_create_tool_with_image	POST /tools/ FormDaten inkl. JPEG-Datei	HTTP 201 image_filename gesetzt	HTTP 201, Dateiname gespeichert	**PASSED**	Multipart-Upload prüfen; static/-Ordner vorhanden?
14	test_upload_tool_image	POST /tools/1/image JPEG-Datei (Multipart)	HTTP 200 image_filename aktualisiert	HTTP 200 erhalten	**PASSED**	Schreibrechte auf static/tool_images/ prüfen
15	test_upload_tool_image_invalid_type	POST /tools/1/image PDF-Datei	HTTP 400 Ungültiger Dateityp	HTTP 400 erhalten	**PASSED**	MIME-Typ-Validierung in Route prüfen
16	test_delete_tool_image	DELETE /tools/1/image	HTTP 200 image_filename = null	HTTP 200, Datei gelöscht	**PASSED**	Dateisystem-Löschlogik und DB-Update prüfen

6 TP-05: Inventar

Quelldatei: src/test/test_tool_items.py | Anzahl Tests: 7

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_tool_item	POST /tool-items/ tool_id, status_id, condition_id (kein inventory_no nötig)	HTTP 201 Exemplar mit auto. INV-Nr.	HTTP 201, INV-0001 generiert	**PASSED**	_next_inventory_no() in CRUD prüfen; Format INV-XXXX korrekt?
2	test_inventory_no_increments	POST /tool-items/ (zweites Mal)	HTTP 201 INV-Nr. = vorherige + 1	HTTP 201, INV-0002 erhalten	**PASSED**	Inkrement-Logik und max()-Query prüfen
3	test_get_tool_items_list	GET /tool-items/	HTTP 200 Inventarliste	HTTP 200 erhalten	**PASSED**	DB-Query und Join auf Tool prüfen
4	test_get_tool_item_by_id	GET /tool-items/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
5	test_get_tool_item_not_found	GET /tool-items/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
6	test_update_tool_item	PUT /tool-items/1 condition_id: 2	HTTP 200	HTTP 200 erhalten	**PASSED**	Update-CRUD prüfen
7	test_delete_tool_item	DELETE /tool-items/1	HTTP 204	HTTP 204 erhalten	**PASSED**	FK-Referenzen in Ausleihen prüfen

7 TP-06: Ausleihen

Quelldatei: src/test/test_loans.py | Anzahl Tests: 8

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_loan	POST /loans/ borrower_id, tool_item_ids	HTTP 201 Ausleihe erstellt	HTTP 201 erhalten	**PASSED**	FK-Referenzen und CRUD-Logik prüfen
2	test_tool_item_status_set_to_loaned	POST /loans/ borrower_id, tool_item_id	HTTP 201 Werkzeugstatus → AUSGELIEHEN	HTTP 201, Status gesetzt	**PASSED**	Status-Update in create_loan prüfen; Status-ID korrekt?
3	test_get_loans_list	GET /loans/	HTTP 200 Ausleihenliste	HTTP 200 erhalten	**PASSED**	DB-Query prüfen
4	test_get_loan_by_id	GET /loans/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
5	test_get_loan_not_found	GET /loans/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
6	test_return_loan	POST /loans/1/return returned_by_user_id items: [{loan_item_id, condition_id}]	HTTP 200 Ausleihe abgeschlossen Werkzeug → VERFÜGBAR	HTTP 200, Status zurückgesetzt	**PASSED**	Return-Logik in CRUD prüfen; condition_id gültig?
7	test_return_loan_twice_rejected	POST /loans/1/return (zweites Mal)	HTTP 400 Bereits zurückgegeben	HTTP 400 erhalten	**PASSED**	Guard in return_loan prüfen; returned_at-Feld gesetzt?
8	test_delete_loan	DELETE /loans/1	HTTP 204	HTTP 204 erhalten	**PASSED**	Kaskadierung auf loan_items prüfen

8 TP-07: Ausleiheanfragen

Quelldatei: src/test/test_loan_requests.py | Anzahl Tests: 6

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_loan_request	POST /loan-requests/ requester_id, tool_item_ids, days_needed	HTTP 201 Anfrage mit Status PENDING	HTTP 201, Status PENDING	**PASSED**	Status-Lookup für PENDING prüfen; tool_item_ids existieren?
2	test_get_loan_requests_list	GET /loan-requests/	HTTP 200 Liste aller Anfragen	HTTP 200 erhalten	**PASSED**	DB-Verbindung und Join-Logik prüfen
3	test_get_loan_request_by_id	GET /loan-requests/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
4	test_decide_loan_request_approve	POST /loan-requests/1/decide approver_id, status_id=APPROVED	HTTP 200 Anfrage genehmigt, Ausleihe erstellt	HTTP 200, Ausleihe erzeugt	**PASSED**	Factory-Funktion create_loan_from_request prüfen; Doppel-Ausleihe-Guard?
5	test_get_loan_request_not_found	GET /loan-requests/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
6	test_delete_loan_request	DELETE /loan-requests/1	HTTP 204	HTTP 204 erhalten	**PASSED**	Kaskadierung auf loan_request_items prüfen

9 TP-08: Meldungen

Quelldatei: src/test/test_tool_item_issues.py | Anzahl Tests: 6

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_tool_item_issue	POST /tool-item-issues/ tool_item_id, reporter_id, title, description	HTTP 201 Meldung erstellt Werkzeugstatus → Wartung Zustand → Verschlissen	HTTP 201, Status und Zustand gesetzt	**PASSED**	Status-Update-Logik in create_issue prüfen; Strategy-Pattern korrekt?
2	test_get_issues_list	GET /tool-item-issues/	HTTP 200 Liste aller Meldungen	HTTP 200 erhalten	**PASSED**	DB-Join-Query prüfen
3	test_get_tool_item_issue_by_id	GET /tool-item-issues/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
4	test_resolve_issue	PUT /tool-item-issues/1 status: Gelöst (kein aktives Ausleihe)	HTTP 200 Werkzeug → Verfügbar Zustand → In Ordnung	HTTP 200, Status zurückgesetzt	**PASSED**	Resolve-Logik prüfen; aktive Ausleihe vorhanden? Guard greift?
5	test_get_issue_not_found	GET /tool-item-issues/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
6	test_delete_issue	DELETE /tool-item-issues/1	HTTP 204	HTTP 204 erhalten	**PASSED**	Kaskadierung prüfen

10 TP-09: Rollen

Quelldatei: src/test/test_roles.py | Anzahl Tests: 8

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_role	POST /roles/ name: Testrolle	HTTP 201	HTTP 201 erhalten	**PASSED**	Unique-Constraint auf name prüfen
2	test_get_roles_list	GET /roles/	HTTP 200	HTTP 200 erhalten	**PASSED**	DB-Abfrage prüfen
3	test_get_role_by_id	GET /roles/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
4	test_get_role_not_found	GET /roles/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
5	test_update_role	PUT /roles/1 name: GeänderteRolle	HTTP 200	HTTP 200 erhalten	**PASSED**	Update-CRUD prüfen
6	test_update_role_not_found	PUT /roles/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler in Update-Route prüfen
7	test_delete_role	DELETE /roles/1	HTTP 204	HTTP 204 erhalten	**PASSED**	FK-Referenzen in Nutzertabelle prüfen
8	test_delete_role_not_found	DELETE /roles/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler in Delete-Route prüfen

11 TP-10: Ausleihepositionen

Quelldatei: src/test/test_loan_items.py | Anzahl Tests: 6

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_get_loan_items	GET /loan-items/	HTTP 200 Liste aller Ausleihpositionen	HTTP 200 erhalten	**PASSED**	DB-Verbindung prüfen; Tabelle loan_items vorhanden?
2	test_get_loan_item_by_id	GET /loan-items/1	HTTP 200 Ausleihposition mit id=1	HTTP 200 erhalten	**PASSED**	ID-Lookup im CRUD prüfen
3	test_get_loan_item_not_found	GET /loan-items/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler in Route prüfen
4	test_create_loan_item	POST /loan-items/ loan_id, tool_item_id	HTTP 201 Neue Ausleihposition	HTTP 201 erhalten	**PASSED**	FK-Referenzen auf loan_id und tool_item_id prüfen
5	test_update_loan_item	PUT /loan-items/1 return_comment: 'OK'	HTTP 200 Aktualisierter Datensatz	HTTP 200 erhalten	**PASSED**	Update-CRUD-Methode debuggen
6	test_delete_loan_item	DELETE /loan-items/1	HTTP 204	HTTP 204 erhalten	**PASSED**	Kaskadierung prüfen; abhängige Datensätze?

12 TP-11: Anfragepositionen

Quelldatei: src/test/test_loan_request_items.py | Anzahl Tests: 6

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_get_loan_request_items	GET /loan-request-items/	HTTP 200 Liste aller Anfragepositionen	HTTP 200 erhalten	**PASSED**	DB-Tabelle loan_request_items prüfen
2	test_get_loan_request_item_by_id	GET /loan-request-items/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
3	test_get_loan_request_item_not_found	GET /loan-request-items/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
4	test_create_loan_request_item	POST /loan-request-items/ request_id, tool_item_id	HTTP 201	HTTP 201 erhalten	**PASSED**	FK-Validierung prüfen
5	test_update_loan_request_item	PUT /loan-request-items/1 tool_item_id: neue ID	HTTP 200	HTTP 200 erhalten	**PASSED**	Update-Logik im CRUD prüfen
6	test_delete_loan_request_item	DELETE /loan-request-items/1	HTTP 204	HTTP 204 erhalten	**PASSED**	Kaskadierung prüfen

13 TP-12: Anfrage-Status

Quelldatei: src/test/test_loan_request_statuses.py | Anzahl Tests: 6

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_loan_request_status	POST /loan-request-statuses/ name: PENDING	HTTP 201 Neuer Status im Body	HTTP 201 erhalten	**PASSED**	Unique-Constraint auf name prüfen
2	test_get_loan_request_statuses	GET /loan-request-statuses/	HTTP 200 Statusliste	HTTP 200 erhalten	**PASSED**	DB-Tabelle prüfen
3	test_get_loan_request_status_by_id	GET /loan-request-statuses/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
4	test_get_loan_request_status_not_found	GET /loan-request-statuses/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
5	test_update_loan_request_status	PUT /loan-request-statuses/1 name: APPROVED	HTTP 200	HTTP 200 erhalten	**PASSED**	Update-CRUD prüfen
6	test_delete_loan_request_status	DELETE /loan-request-statuses/1	HTTP 204	HTTP 204 erhalten	**PASSED**	FK-Referenzen in Anfragen prüfen

14 TP-13: Werkzeugstatus

Quelldatei: src/test/test_tool_statuses.py | Anzahl Tests: 6

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_tool_status	POST /tool-statuses/ name: Verfügbar	HTTP 201	HTTP 201 erhalten	**PASSED**	Unique-Constraint prüfen
2	test_get_tool_statuses	GET /tool-statuses/	HTTP 200	HTTP 200 erhalten	**PASSED**	DB-Abfrage prüfen
3	test_get_tool_status_by_id	GET /tool-statuses/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
4	test_get_tool_status_not_found	GET /tool-statuses/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
5	test_update_tool_status	PUT /tool-statuses/1 name: Ausgeliehen	HTTP 200	HTTP 200 erhalten	**PASSED**	Update-Logik prüfen
6	test_delete_tool_status	DELETE /tool-statuses/1	HTTP 204	HTTP 204 erhalten	**PASSED**	FK-Referenzen in Exemplaren prüfen

15 TP-14: Werkzeugzustände

Quelldatei: src/test/test_tool_conditions.py | Anzahl Tests: 6

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_tool_condition	POST /tool-conditions/ name: In Ordnung	HTTP 201	HTTP 201 erhalten	**PASSED**	Unique-Constraint prüfen
2	test_get_tool_conditions	GET /tool-conditions/	HTTP 200	HTTP 200 erhalten	**PASSED**	DB-Abfrage prüfen
3	test_get_tool_condition_by_id	GET /tool-conditions/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
4	test_get_tool_condition_not_found	GET /tool-conditions/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
5	test_update_tool_condition	PUT /tool-conditions/1 name: Verschlissen	HTTP 200	HTTP 200 erhalten	**PASSED**	Update-Logik prüfen
6	test_delete_tool_condition	DELETE /tool-conditions/1	HTTP 204	HTTP 204 erhalten	**PASSED**	Kaskadierung prüfen

16 TP-15: Meldungs-Status

Quelldatei: src/test/test_tool_item_issue_statuses.py | Anzahl Tests: 6

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	test_create_issue_status	POST /tool-item-issue-statuses/ name: Offen	HTTP 201	HTTP 201 erhalten	**PASSED**	Unique-Constraint prüfen
2	test_get_issue_statuses	GET /tool-item-issue-statuses/	HTTP 200	HTTP 200 erhalten	**PASSED**	DB-Abfrage prüfen
3	test_get_issue_status_by_id	GET /tool-item-issue-statuses/1	HTTP 200	HTTP 200 erhalten	**PASSED**	ID-Lookup prüfen
4	test_get_issue_status_not_found	GET /tool-item-issue-statuses/9999	HTTP 404	HTTP 404 erhalten	**PASSED**	404-Handler prüfen
5	test_update_issue_status	PUT /tool-item-issue-statuses/1 name: Gelöst	HTTP 200	HTTP 200 erhalten	**PASSED**	Update-CRUD prüfen
6	test_delete_issue_status	DELETE /tool-item-issue-statuses/1	HTTP 204	HTTP 204 erhalten	**PASSED**	FK-Referenzen in Issues prüfen

17 Frontend-Tests (Playwright / End-to-End)

Die folgenden Tests wurden mit Playwright (Chromium) gegen die React-Anwendung ausgeführt. Alle API-Aufrufe wurden über page.route()-Mocks abgefangen, sodass kein laufendes Backend benötigt wird.

17.1 FE-01: Authentifizierung

Quelldatei: tests/auth.spec.ts | Anzahl Tests: 8

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	Login-Seite wird angezeigt	GET /login (kein Token im localStorage)	Login-Formular mit E-Mail- und Passwortfeld sichtbar	Formular sichtbar	**PASSED**	Route-Guard prüfen; AuthContext korrekt initialisiert?
2	Ohne Token wird auf /login weitergeleitet	GET /dashboard (kein Token vorhanden)	Weiterleitung auf /login	URL = /login	**PASSED**	PrivateRoute-Komponente prüfen; localStorage-Abfrage korrekt?
3	Erfolgreicher Login leitet zum Dashboard weiter	POST /auth/login email: admin@firma.de password: admin123	HTTP 200, Token gesetzt Weiterleitung auf /dashboard	URL = /dashboard	**PASSED**	API-Mock prüfen; navigate()-Aufruf nach Login korrekt?
4	Falsche Zugangsdaten zeigen Fehlermeldung	POST /auth/login email: falsch@firma.de password: falsch	HTTP 401 Fehlermeldung im UI sichtbar	Fehlermeldung sichtbar	**PASSED**	Error-Handling im Login-Handler prüfen; Toast oder Alert vorhanden?
5	Leere Felder verhindern Absenden	Klick auf Login-Button ohne Eingaben	Formular nicht abgesendet bzw. Validierungsmeldung	Absenden verhindert	**PASSED**	HTML5-required-Attribut oder Frontend-Validierung prüfen
6	Logout löscht Token und leitet zu /login	Klick auf Logout-Button (Token im localStorage)	Token entfernt Weiterleitung auf /login	URL = /login	**PASSED**	logout()-Funktion im AuthContext prüfen; localStorage.removeItem aufgerufen?
7	Mitarbeiter-Token gewährt Zugang zum Dashboard	localStorage: Employee-JWT GET /dashboard	Dashboard sichtbar	Dashboard sichtbar	**PASSED**	JWT-Dekodierung in AuthContext prüfen; role_id korrekt ausgelesen?
8	Admin-Only-Route /users ist für Mitarbeiter gesperrt	localStorage: Employee-JWT GET /users	URL ≠ /users (Weiterleitung)	Weiterleitung erfolgt	**PASSED**	AdminRoute / require_role-Guard im Frontend prüfen

17.2 FE-02: Dashboard & Navigation

Quelldatei: tests/dashboard.spec.ts | Anzahl Tests: 11

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	Dashboard lädt ohne Fehler	GET /dashboard (Admin-Token)	Seite lädt, kein Fehler im UI	Keine Fehler	**PASSED**	API-Mocks prüfen; alle Fetch-Aufrufe gecacht?
2	Sidebar-Navigation ist sichtbar	GET /dashboard	nav- oder aside-Element vorhanden	Navigation sichtbar	**PASSED**	Sidebar-Komponente prüfen; CSS display:none?
3	Dashboard-Seite enthält KPI-Karten	GET /dashboard	Mindestens eine Statistik-Karte sichtbar	KPI-Karte sichtbar	**PASSED**	DashboardPage-Datenquelle prüfen; API-Antwort leer?
4	Admin sieht Nutzerverwaltung in der Sidebar	GET /dashboard (Admin-Token)	Link 'Nutzer' in Sidebar sichtbar	Link sichtbar	**PASSED**	Sidebar-Rollenprüfung prüfen; isAdmin korrekt?
5	Admin sieht Rollenverwaltung in der Sidebar	GET /dashboard (Admin-Token)	Link 'Rollen' in Sidebar sichtbar	Link sichtbar	**PASSED**	Sidebar-Bedingung für Rollen-Link prüfen
6	Navigation zu Werkzeugseite funktioniert	Klick auf 'Werkzeuge' in Sidebar	URL = /tools	URL = /tools	**PASSED**	React Router Link-Pfad prüfen
7	Navigation zu Inventarseite funktioniert	Klick auf 'Inventar' in Sidebar	URL = /inventory	URL = /inventory	**PASSED**	React Router Link-Pfad prüfen
8	Navigation zu Ausleihen funktioniert	Klick auf 'Ausleihen' in Sidebar	URL = /loans	URL = /loans	**PASSED**	React Router Link-Pfad prüfen
9	Navigation zu Anfragen funktioniert	Klick auf 'Anfragen' in Sidebar	URL = /loan-requests	URL = /loan-requests	**PASSED**	React Router Link-Pfad prüfen
10	Navigation zu Meldungen funktioniert	Klick auf 'Meldungen' in Sidebar	URL = /issues	URL = /issues	**PASSED**	React Router Link-Pfad prüfen
11	Mitarbeiter sieht keine Admin-Links	GET /dashboard (Employee-Token)	Kein 'Nutzer'- oder 'Rollen'-Link in Sidebar	Admin-Links nicht sichtbar	**PASSED**	Sidebar-Rollenprüfung prüfen; role_id aus Token korrekt?

17.3 FE-03: Werkzeuge

Quelldatei: tests/tools.spec.ts | Anzahl Tests: 11

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	Werkzeuge werden angezeigt	GET /tools (Mock: Hammer, Bohrmaschine)	'Hammer' und 'Bohrmaschine' sichtbar	Beide sichtbar	**PASSED**	API-Mock für /gettools prüfen; tool_name-Feld korrekt?
2	Kategorie wird angezeigt	GET /tools	Kategoriename 'Handwerkzeug' sichtbar	Sichtbar	**PASSED**	category.name aus API-Antwort prüfen
3	Suche filtert Werkzeuge	Eingabe 'Hammer' in Suchfeld	'Hammer' weiterhin sichtbar	Sichtbar	**PASSED**	Suchfilter-Logik in ToolsPage prüfen
4	"Neues Werkzeug"-Button öffnet Dialog	Klick auf 'Neues Werkzeug'	Dialog (role=dialog) sichtbar	Dialog sichtbar	**PASSED**	Dialog-State (useState) prüfen
5	Pflichtfelder im Erstellungsdialog sind vorhanden	Klick auf 'Neues Werkzeug' Dialog öffnen	Text-Input für Name vorhanden	Input sichtbar	**PASSED**	Dialog-Formular prüfen; Input nicht als disabled?
6	Werkzeug erstellen – erfolgreich	POST /tools/ name: Schraubenzieher category_id: 1	HTTP 201 API aufgerufen	API aufgerufen	**PASSED**	createtool-Endpoint in services.ts prüfen; Payload korrekt?
7	Klick auf Werkzeug öffnet Detaildialog	Klick auf 'Hammer' in Liste	Dialog mit Werkzeugdetails sichtbar	Dialog sichtbar	**PASSED**	onClick-Handler in ToolCard prüfen
8	Detaildialog zeigt Werkzeugname	Klick auf 'Hammer' Dialog öffnen	'Hammer' im Dialog sichtbar	Sichtbar	**PASSED**	Dialog-Props prüfen; tool_name korrekt übergeben?
9	Dialog kann geschlossen werden	Klick auf Hammer, dann Escape	Dialog nicht mehr sichtbar	Dialog geschlossen	**PASSED**	onOpenChange/onClose-Handler prüfen
10	Kategoriefilter zeigt nur passende Werkzeuge	Kategorie 'Handwerkzeug' auswählen	'Bohrmaschine' nicht sichtbar	Gefiltert	**PASSED**	Filterlogik in ToolsPage prüfen
11	Werkzeug löschen – Bestätigungsdialog erscheint	Klick auf Löschen in Detaildialog	Bestätigungsbutton sichtbar	Bestätigung sichtbar	**PASSED**	Delete-Confirm-Dialog prüfen; confirm()-Aufruf?

17.4 FE-04: Inventar / Exemplare

Quelldatei: tests/inventory.spec.ts | Anzahl Tests: 11

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	Inventarliste wird angezeigt	GET /inventory (Mock: INV-0001)	'INV-0001' sichtbar	Sichtbar	**PASSED**	API-Mock für /gettoolitems prüfen; inventory_no-Feld?
2	Werkzeugnamen sind sichtbar	GET /inventory	'Hammer' in Liste sichtbar	Sichtbar	**PASSED**	tool.tool_name-Feld in Mock prüfen
3	Status-Badges sind sichtbar	GET /inventory	Status 'Verfügbar' oder Badge sichtbar	Badge sichtbar	**PASSED**	labels.ts AVAILABLE-Mapping prüfen
4	Suche nach Inventarnummer funktioniert	Eingabe 'INV-0001' in Suchfeld	INV-0001-Zeile sichtbar	Sichtbar	**PASSED**	Suchfilter inventory_no.includes() prüfen
5	Suche nach Werkzeugname funktioniert	Eingabe 'Hammer' in Suchfeld	Hammer-Zeile sichtbar	Sichtbar	**PASSED**	Suchfilter tool.tool_name.includes() prüfen
6	"Neues Exemplar"-Button öffnet Dialog	Klick auf 'Neues Exemplar'	Dialog sichtbar	Dialog sichtbar	**PASSED**	Dialog-State prüfen
7	Erstellungsdialog enthält Werkzeug-Auswahl	Dialog öffnen	Combobox/Select für Werkzeug vorhanden	Auswahl vorhanden	**PASSED**	Dialog-Formular prüfen
8	Exemplar erstellen – API wird aufgerufen	POST /tool-items/ tool_id, status_id, condition_id	API createtoolitem aufgerufen	API aufgerufen	**PASSED**	createtoolitem-Endpoint prüfen; Payload korrekt?
9	Klick auf Tabellenzeile öffnet Detaildialog	Klick auf erste Tabellenzeile	Dialog sichtbar	Dialog sichtbar	**PASSED**	onClick auf tr prüfen
10	Zustandsbadge "In Ordnung" ist grün/sichtbar	GET /inventory (Mock: condition OK)	'In Ordnung' oder 'OK' sichtbar	Sichtbar	**PASSED**	labels.ts OK-Mapping prüfen
11	Zustandsbadge "Abgenutzt" ist sichtbar	GET /inventory (Mock: condition WORN)	'Abgenutzt' sichtbar	Sichtbar	**PASSED**	labels.ts WORN→'Abgenutzt' prüfen

17.5 FE-05: Ausleiheanfragen

Quelldatei: tests/loan-requests.spec.ts | Anzahl Tests: 10

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	Anfragen werden angezeigt	GET /loan-requests (Mock: REQUESTED-Status)	Eva, Hammer oder Status sichtbar	Sichtbar	**PASSED**	API-Mock getloanrequests prüfen
2	Status-Badge ist sichtbar	GET /loan-requests	'Ausstehend' oder 'REQUESTED' sichtbar	Badge sichtbar	**PASSED**	labels.ts REQUESTED-Mapping prüfen
3	Statusfilter ist vorhanden	GET /loan-requests	Tab, Button oder Select für Filter vorhanden	Filter vorhanden	**PASSED**	Filterkomponente prüfen
4	"Neue Anfrage"-Button öffnet Dialog	Klick auf 'Neue Anfrage'	Dialog sichtbar	Dialog sichtbar	**PASSED**	Dialog-State prüfen
5	Anfrageformular enthält Felder für Werkzeug und Dauer	Dialog öffnen	Zahlen-Input oder Combobox vorhanden	Felder vorhanden	**PASSED**	Dialog-Formular prüfen; days_needed + tool_id?
6	Neue Anfrage erstellen – API wird aufgerufen	POST /createloanrequest days_needed: 3 tool_id: 1	API aufgerufen (HTTP 201)	API aufgerufen	**PASSED**	handleCreate() in LoanRequestsPage prüfen
7	Klick auf Anfrage öffnet Detaildialog	Klick auf Anfrage-Zeile	Dialog sichtbar	Dialog sichtbar	**PASSED**	onClick-Handler prüfen
8	Admin kann Anfrage genehmigen	Klick auf 'Genehmigen' im Dialog	API decideloanrequest aufgerufen	API aufgerufen	**PASSED**	handleDecide() prüfen; status_id APPROVED?
9	Admin kann Anfrage ablehnen	Klick auf 'Ablehnen' im Dialog	API decideloanrequest aufgerufen	API aufgerufen	**PASSED**	handleDecide() prüfen; status_id REJECTED?
10	Leere Anfragenliste zeigt Hinweistext	Mock: leere Liste GET /loan-requests	'Keine Anfragen' oder ähnlich sichtbar	Hinweistext sichtbar	**PASSED**	Empty-State-Rendering in LoanRequestsPage prüfen

17.6 FE-06: Ausleihen

Quelldatei: tests/loans.spec.ts | Anzahl Tests: 10

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	Ausleihliste wird angezeigt	GET /loans (Mock: Eva / INV-0001)	Eva, INV-0001 oder Hammer sichtbar	Sichtbar	**PASSED**	API-Mock getloans prüfen
2	Aktivfilter-Tabs sind sichtbar	GET /loans	Tab/Button für Alle, Aktiv, Überfällig, Zurückgegeben	Tabs sichtbar	**PASSED**	Filter-Tabs-Komponente prüfen
3	Seiteninhalt wird geladen	GET /loans	'Ausleihen'-Text sichtbar	Sichtbar	**PASSED**	LoansPage-Rendering prüfen
4	Tabellenheader sind sichtbar	GET /loans	'Ausleiher', 'Exemplare' oder 'Ausgegeben' sichtbar	Header sichtbar	**PASSED**	Tabellen-Header in LoansPage prüfen
5	Klick auf Ausleihe öffnet Detaildialog	Klick auf erste Tabellenzeile	Dialog sichtbar	Dialog sichtbar	**PASSED**	onClick auf tr prüfen
6	Rückgabe-Button ist im Dialog sichtbar (aktive Ausleihe)	Dialog aktiver Ausleihe öffnen	'Rückgabe'- oder 'Zurückgeben'-Button sichtbar	Button sichtbar	**PASSED**	Bedingung für Rückgabe-Button prüfen; returned_at null?
7	Rückgabe-Dialog öffnet sich	Klick auf Rückgabe-Button	Dialog sichtbar	Dialog sichtbar	**PASSED**	Return-Dialog-State prüfen
8	Rückgabe – Dialog hat Bestätigen-Button	Return-Dialog öffnen	Bestätigen/Speichern-Button sichtbar	Button sichtbar	**PASSED**	Return-Form prüfen; Submit-Button vorhanden?
9	Suche nach Inventarnummer filtert Ausleihen	Eingabe 'INV-0001' in Suchfeld	Filterung ohne Fehler	Kein Fehler	**PASSED**	Suchfilter in LoansPage prüfen
10	Filter "Aktiv" zeigt nur offene Ausleihen	Klick auf 'Aktiv'-Tab	Nur aktive Ausleihen sichtbar	Gefiltert	**PASSED**	Filter-Logik (returned_at === null) prüfen

17.7 FE-07: Meldungen (Issues)

Quelldatei: tests/issues.spec.ts | Anzahl Tests: 14

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	Meldungsliste wird angezeigt	GET /issues (Mock: 'Griff gebrochen')	'Griff gebrochen' sichtbar	Sichtbar	**PASSED**	API-Mock gettoolitemissues prüfen
2	Inventarnummer der Meldung ist sichtbar	GET /issues	'INV-0001' sichtbar	Sichtbar	**PASSED**	tool_item.inventory_no in Mock prüfen
3	Melder ist sichtbar	GET /issues	'Eva' oder 'emp@firma' sichtbar	Sichtbar	**PASSED**	reporter-Feld in Mock prüfen
4	Status-Badge ist sichtbar	GET /issues	'Offen', 'Open' oder 'In Bearbeitung' sichtbar	Badge sichtbar	**PASSED**	labels.ts OPEN-Mapping prüfen
5	Statusfilter ist vorhanden	GET /issues	Combobox, Tab oder Select vorhanden	Filter vorhanden	**PASSED**	Filterkomponente in IssuesPage prüfen
6	Suche nach Titel funktioniert	Eingabe 'Griff' in Suchfeld	'Griff gebrochen' weiterhin sichtbar	Sichtbar	**PASSED**	Suchfilter title.includes() prüfen
7	"Neue Meldung"-Button öffnet Dialog	Klick auf 'Neue Meldung'	Dialog sichtbar	Dialog sichtbar	**PASSED**	Dialog-State prüfen
8	Meldeformular enthält Titel-Feld	Dialog öffnen	Text-Input für Titel vorhanden	Input vorhanden	**PASSED**	Dialog-Formular prüfen
9	Neue Meldung erstellen – API wird aufgerufen	POST /createtoolitemissue title, tool_item_id	API aufgerufen (HTTP 201)	API aufgerufen	**PASSED**	handleCreate() in IssuesPage prüfen
10	Klick auf Meldung öffnet Detaildialog	Klick auf 'Griff gebrochen'	Dialog sichtbar	Dialog sichtbar	**PASSED**	onClick-Handler prüfen
11	Detaildialog zeigt Beschreibung	Dialog öffnen	Beschreibungstext sichtbar	Sichtbar	**PASSED**	Dialog-Props prüfen; description korrekt übergeben?
12	Admin sieht Aktions-Buttons in der Meldungsliste	GET /issues (Admin-Token)	Button in Tabellenzeile sichtbar	Button sichtbar	**PASSED**	Rollenprüfung für Aktions-Buttons prüfen
13	Mitarbeiter sieht keine Bearbeiten-Schaltfläche	GET /issues (Employee-Token)	Seite lädt, Bearbeiten-Button nicht sichtbar	Seite sichtbar	**PASSED**	Rollenbedingte Anzeige prüfen
14	Meldung löschen – API wird aufgerufen	Klick Löschen → Bestätigen	API deletetoolitemissue aufgerufen	API aufgerufen	**PASSED**	handleDelete() und Confirm-Dialog prüfen

17.8 FE-08: Nutzerverwaltung

Quelldatei: tests/users.spec.ts | Anzahl Tests: 12

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	Nutzertabelle wird angezeigt	GET /users (Mock: Anna, Max, Eva)	Anna, Max und Eva sichtbar	Alle sichtbar	**PASSED**	API-Mock getusers prüfen; firstname-Feld?
2	E-Mail-Adressen sind sichtbar	GET /users	'admin@firma.de' sichtbar	Sichtbar	**PASSED**	email-Feld in Mock prüfen
3	Rollenbadges sind sichtbar	GET /users	'Admin'-Badge oder Text sichtbar	Sichtbar	**PASSED**	role.name in Mock prüfen
4	"Neuer Nutzer"-Button öffnet Dialog	Klick auf 'Neuer Nutzer'	Dialog sichtbar	Dialog sichtbar	**PASSED**	Dialog-State prüfen
5	Erstellungsformular enthält E-Mail-Feld	Dialog öffnen	Text-Input für E-Mail vorhanden	Input vorhanden	**PASSED**	Dialog-Formular prüfen
6	Erstellungsformular enthält Passwortfeld	Dialog öffnen	input[type='password'] vorhanden	Passwortfeld vorhanden	**PASSED**	Dialog-Formular prüfen
7	Nutzer erstellen – API wird aufgerufen	POST /createuser Vorname, Nachname Email, Passwort, Rolle	API aufgerufen (HTTP 201)	API aufgerufen	**PASSED**	handleCreate() in UsersPage prüfen
8	Klick auf Tabellenzeile öffnet Detaildialog	Klick auf erste Tabellenzeile	Dialog sichtbar	Dialog sichtbar	**PASSED**	onClick auf tr prüfen
9	Detaildialog zeigt E-Mail	Tabellenzeile klicken	E-Mail mit 'firma.de' im Dialog sichtbar	Sichtbar	**PASSED**	Dialog-Props prüfen
10	Nutzer bearbeiten – Dialog öffnet sich	Detaildialog öffnen Klick auf 'Bearbeiten'	Bearbeitungsdialog sichtbar	Dialog sichtbar	**PASSED**	Edit-Dialog-State prüfen
11	Nutzer löschen – API wird aufgerufen	Klick auf Löschen-Button Bestätigen	API deleteuser aufgerufen	API aufgerufen	**PASSED**	handleDelete() und Confirm prüfen
12	Mitarbeiter kann /users nicht aufrufen	GET /users (Employee-Token)	URL ≠ /users (Weiterleitung)	Weiterleitung erfolgt	**PASSED**	AdminRoute-Guard prüfen

17.9 FE-09: Abteilungen

Quelldatei: tests/departments.spec.ts | Anzahl Tests: 12

Nr.	Testfall	Eingabe	Erwartete Ausgabe	Ergebnis	Status	Maßnahmen bei Fehlschlag
1	Abteilungen werden als Karten angezeigt	GET /departments (Mock: Montage, Lager)	Abteilungsnamen als Karten sichtbar	Karten sichtbar	**PASSED**	API-Mock getdepartments prüfen
2	"Neue Abteilung"-Button ist für Admin sichtbar	GET /departments (Admin-Token)	'Neue Abteilung'-Button sichtbar	Button sichtbar	**PASSED**	Rollenprüfung für Button prüfen
3	"Neue Abteilung"-Dialog öffnet sich	Klick auf 'Neue Abteilung'	Dialog sichtbar	Dialog sichtbar	**PASSED**	Dialog-State prüfen
4	Erstellungsformular enthält Namensfeld	Dialog öffnen	Text-Input vorhanden	Input vorhanden	**PASSED**	Dialog-Formular prüfen
5	Erstellungsformular enthält Leiter-Dropdown	Dialog öffnen	Combobox für Leiter vorhanden	Combobox vorhanden	**PASSED**	lead_user_id-Auswahl prüfen
6	Abteilung erstellen – API wird aufgerufen	POST /createdepartment name: Testmontage	API aufgerufen (HTTP 201)	API aufgerufen	**PASSED**	handleCreate() in DepartmentsPage prüfen
7	Klick auf Abteilung öffnet Detaildialog	Klick auf Abteilungskarte	Dialog sichtbar	Dialog sichtbar	**PASSED**	onClick-Handler prüfen
8	Bearbeiten-Button öffnet Bearbeitungsdialog	Detaildialog öffnen Klick auf 'Bearbeiten'	Bearbeitungsdialog sichtbar	Dialog sichtbar	**PASSED**	Edit-Dialog-State prüfen
9	Abteilung bearbeiten – API wird aufgerufen	PUT /updatedepartment/<id> name: geändert	API aufgerufen	API aufgerufen	**PASSED**	handleUpdate() prüfen
10	Abteilung löschen – API wird aufgerufen	Klick auf Löschen Bestätigen	API deletedepartment aufgerufen	API aufgerufen	**PASSED**	handleDelete() und Confirm prüfen
11	Mitarbeiter kann /departments nicht aufrufen	GET /departments (Employee-Token)	Weiterleitung auf /dashboard	Weiterleitung erfolgt	**PASSED**	Rollengeschützte Route prüfen
12	Abteilungsleiter kann /departments aufrufen	GET /departments (Manager-Token)	Seite lädt ohne Weiterleitung	Seite sichtbar	**PASSED**	Rollenprüfung: MANAGER oder ADMIN erlaubt?